

PRACTICAL - 7

Implementation of Making Change Problem using Dynamic Programming

Aim of the Practical:

To find the minimum number of coins required to make a target amount using dynamic programming and analyze its complexity.

Algorithm:

1. Initialize $dp[0] = 0$ and $dp[1...A] = \infty$.
2. For each amount x from 1 to A , try every coin denomination.
3. If $coin \leq x$ and $dp[x-coin]$ is finite, update $dp[x] = \min(dp[x], dp[x-coin] + 1)$.
4. Return $dp[A]$, or -1 if it is still infinite.

Program:

```
#include <iostream>
#include <vector>
#include <climits>
#include <algorithm>
using namespace std;
int minCoins(const vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, INT_MAX);
    dp[0] = 0;
    for (int x = 1; x <= amount; x++) {
        for (int coin : coins) {
            if (coin <= x && dp[x - coin] != INT_MAX) {
                dp[x] = min(dp[x], dp[x - coin] + 1);
            }
        }
    }
    return dp[amount] == INT_MAX ? -1 : dp[amount];
}
int main() {
    vector<int> coins = {1, 3, 4};
    int amount = 6;
    cout << minCoins(coins, amount);
}
```

Output:

2

Step-by-Step DP for coins {1,3,4}, amount 6:

- $dp[0] = 0$
- $dp[1] = 1$ using {1}
- $dp[2] = 2$ using {1,1}
- $dp[3] = 1$ using {3}
- $dp[4] = 1$ using {4}
- $dp[5] = 2$ using {4,1}
- $dp[6] = 2$ using {3,3}

Time Analysis:

Derivation of Time Complexity:

Let A be the target amount and m be the number of coin denominations.

The outer loop runs once for each amount from 1 to A .

Outer-loop iterations = A

For every amount, the algorithm checks all m coin denominations.

Inner-loop iterations per amount = m

Each inner-loop operation performs a constant-time comparison and possible update.

$$T(A,m) = A \cdot m \cdot O(1)$$

$$T(A,m) = \Theta(Am)$$

The dp array contains $A + 1$ entries.

$$S(A) = \Theta(A)$$

Final Time Analysis:

- Time Complexity: $\Theta(Am)$.
- Space Complexity: $\Theta(A)$.

Result:

The Making Change problem was solved successfully using dynamic programming, and the minimum number of coins for amount 6 is 2.